

Unit-II
Lecture: 2
Binary Search

Binary search is an efficient algorithm for finding an item from a sorted list of items.

In binary search, we **search a sorted array by repeatedly dividing the search interval in half**, until we have narrowed down the possible locations to just one.

- **Binary search works only on a sorted set of elements.**
- i.e. to use binary search on a collection, **the collection must first be sorted.**

Binary search has run time complexity of $O(\log n)$. This search algorithm works on the principle of divide and conquer.

Implementing Binary Search Algorithm:

1. Start with the middle element:
 - If the target value is equal to the middle element of the array, then return the index of the middle element.
 - If not, then compare the middle element with the target value,
 - If the target value is greater than the number in the middle index, then pick the elements to the right of the middle index, and start with Step 1.
 - If the target value is less than the number in the middle index, then pick the elements to the left of the middle index, and start with Step 1.
2. When a match is found, return the index of the element matched.
3. If no match is found, then return -1.

How to find the middle element:

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$



Example:

The following is sorted array and let us assume that we need to search the location of value 31 using binary search:

10	14	19	26	27	31	33	35	42	44
0	1	2	3	4	5	6	7	8	9

First, we shall determine half of the array by using this formula –

$$\text{mid} = \text{low} + (\text{high} - \text{low}) / 2$$

Here it is, $0 + (9 - 0) / 2 = 4$ (integer value of 4.5). So, 4 is the mid of the array.

10	14	19	26	27	31	33	35	42	44
0	1	2	3	4	5	6	7	8	9

Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.

10	14	19	26	27	31	33	35	42	44
0	1	2	3	4	5	6	7	8	9

We change our low to mid + 1 and find the new mid value again.

$$\begin{aligned} \text{low} &= \text{mid} + 1 \\ \text{mid} &= \text{low} + (\text{high} - \text{low}) / 2 \end{aligned}$$

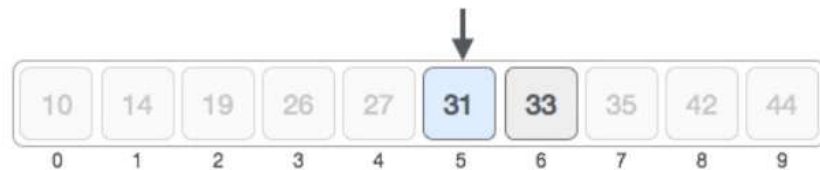
Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.



The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.



Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.



We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.